

AGRICONNECT

Farm Management Platform

Software Requirements Specification (SRS)

Document Field	Details
Document Title	Software Requirements Specification — Agriconnect
Version	1.0
Date	April 2026
Status	Final Draft
Technology Stack	C++ Backend (Drogon) React Native Frontend
Architecture	REST API PostgreSQL OpenCV Image Analysis
Prepared For	Agriconnect Development Team

Connecting Farmers · Wholesalers · Transporters

Table of Contents

Table of Contents.....	2
1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions, Acronyms, and Abbreviations.....	4
1.4 Overview	5
2. Overall System Description	6
2.1 Product Perspective	6
2.2 System Actors	6
2.3 Product Functions Summary.....	6
2.4 User Characteristics	7
2.5 Constraints	7
2.6 Assumptions and Dependencies	7
3. Functional Requirements	8
3.1 User Management & Authentication	8
3.1.1 Registration.....	8
3.1.2 Authentication	8
3.2 Produce Listing & Marketplace	8
3.2.1 Farmer Listing Management	8
3.2.2 Wholesaler Browse & Purchase	9
3.3 Crop Health Analysis	9
3.3.1 Image Upload & Analysis Request	9
3.4 Transport Coordination	10
3.4.1 Transport Job Management	10
3.5 Notification System.....	11
3.6 Farmer Education Module	11
4. Non-Functional Requirements.....	Error! Bookmark not defined.
4.1 Performance Requirements.....	12
4.2 Security Requirements	12
4.3 Usability Requirements.....	12
4.4 Reliability and Availability	13
4.5 Maintainability & Scalability	13
5. UML Diagrams	14
5.1 Use Case Diagram	14
5.2 Sequence Diagram — Crop Analysis Flow	15

5.3 Class Diagram — Agriconnect Backend	17
5.3.1 Enum: UserType	18
5.3.2 Class Definitions	18
5.3.3 Class Relationships Summary.....	18
6. System Architecture	19
6.1 Architectural Overview.....	19
6.2 Backend Architecture (C++ Drogon)	19
6.3 Frontend Architecture (React Native).....	19
6.4 API Endpoint Summary	20
7. Data Requirements	21
7.1 Database Schema Overview	21
7.1.1 users Table	21
7.1.2 listings Table	21
7.1.3 crop_analyses Table	21
7.1.4 transport_jobs Table	22
8. External Interface Requirements	23
8.1 User Interfaces	23
8.2 Hardware Interfaces	23
8.3 Software Interfaces.....	23
9. Testing & Quality Assurance Requirements	24
9.1 Unit Testing	24
9.2 Integration Testing.....	24
9.3 User Acceptance Testing.....	24
10. Constraints, Compliance & Regulatory Requirements.....	25
10.1 Technical Constraints	25
10.2 Regulatory Compliance	25
11. Appendices	25
Appendix A — Use Case Reference Summary.....	25
Appendix B — Revision History	26
Appendix C — References	26

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for the Agriconnect Farm Management Platform. The platform is a mobile and web-based ecosystem designed to digitally connect farmers, wholesalers, and transporters in order to streamline agricultural commerce, enable modern crop health analysis, and reduce post-harvest losses through coordinated logistics.

This document is intended for software developers, system architects, product managers, quality assurance engineers, and stakeholders involved in the design, development, testing, and deployment of the Agriconnect system.

1.2 Scope

The Agriconnect Platform encompasses the following major subsystems:

- **Agricultural Marketplace** A digital marketplace enabling farmers to list produce for sale and wholesalers to browse and purchase directly from farms.
- **Crop Health Analysis Engine** An AI-powered image analysis module, built in C++ using OpenCV, that accepts crop images from farmers and returns health assessments using texture-based feature extraction and machine learning prediction models.
- **Transport Coordination System** A logistics module that links farmers needing produce transport from farm to market with available transporters, enabling job posting, matching, and acceptance.
- **User Management & Authentication** Secure registration, login, and role-based access control for all actor types: Farmers, Wholesalers, and Transporters.
- **Educational Farming Content** A knowledge base providing modern farming techniques and agronomic guidance to registered farmers.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer
C++	Programming language used for the backend (Drogon framework)
React Native	Cross-platform mobile frontend framework by Meta
OpenCV	Open Source Computer Vision library used for image processing
PostgreSQL	Open-source relational database management system
Drogon	High-performance C++ HTTP/WebSocket framework
JWT	JSON Web Token — used for stateless authentication

RBAC	Role-Based Access Control
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case

1.4 Overview

The remainder of this document is organized as follows: Section 2 provides an overall system description including context, actors, and constraints. Section 3 details functional requirements organized by system module. Section 4 specifies non-functional requirements. Section 5 presents UML diagrams including Use Case, Sequence, and Class Diagrams. Section 6 describes system architecture and technology stack. Section 7 covers data requirements and Section 8 outlines API specifications.

2. Overall System Description

2.1 Product Perspective

Agriconnect is an independent, standalone platform built as a mobile-first application (React Native) backed by a high-performance C++ API server (Drogon). It integrates a native C++ image processing engine (OpenCV) to analyze crop health from photographs. The system communicates via RESTful JSON APIs and persists all data in a PostgreSQL relational database.

The platform serves as a digital agricultural hub bridging the gap between rural farmers who lack market access, urban wholesalers seeking reliable produce, and transporters looking for logistics work in the agricultural supply chain.

2.2 System Actors

Actor	Type	Description	Key Capabilities
Farmer	Primary Human	A registered agricultural producer who manages crops and needs market access.	List produce, analyze crops, request transport, view market prices
Wholesaler	Primary Human	A buyer who purchases produce in bulk from multiple farmers.	Browse listings, place bulk orders, request delivery transport
Transporter	Primary Human	An individual or fleet owner providing farm-to-market logistics.	View transport jobs, accept delivery assignments, update delivery status
C++ Analysis Engine	System Actor	Automated image processing component triggered by crop upload events.	Extract texture features, run prediction model, return health status
Administrator	Secondary Human	Platform administrator managing users, listings, and system health.	Manage all users, moderate listings, view analytics

2.3 Product Functions Summary

- User registration and secure login with role differentiation (Farmer, Wholesaler, Transporter)
- Farmer dashboard: produce listing management, transport requests, crop analysis portal
- Wholesaler dashboard: produce browsing, search/filter by crop type and region, purchase requests
- Transporter dashboard: available transport jobs, job acceptance, delivery tracking
- Crop image upload for automated health analysis using C++ OpenCV engine
- Health prediction result display with confidence score and agri-recommendations
- Modern farming education section accessible to all farmers
- In-app notifications for job matches, purchase requests, and analysis results

2.4 User Characteristics

The target user base primarily includes low-to-medium tech-literacy agricultural stakeholders in sub-Saharan African markets. The mobile frontend must be optimized for:

- Low-bandwidth environments (2G/3G mobile networks)
- Offline-first design patterns where feasible
- Multi-language support (French and English as primary languages for Cameroon market)
- Simple, icon-driven UI requiring minimal reading

2.5 Constraints

- Backend must be implemented in C++ using the Drogon framework for performance requirements.
- Mobile frontend must be implemented using React Native for cross-platform iOS and Android support.
- Image analysis must use OpenCV for texture feature extraction; no third-party ML cloud services.
- All data must be stored in PostgreSQL; no NoSQL databases for primary data.
- All API communication must use HTTPS with JWT-based authentication.
- The system must comply with applicable data protection regulations for user personal data.

2.6 Assumptions and Dependencies

- Farmers possess smartphones with camera capability and a minimum of 2G internet connectivity.
- The Drogon C++ framework version 1.9+ is used for all backend development.
- React Native version 0.73+ is used for mobile development.
- OpenCV 4.8+ is integrated as a statically linked library in the C++ backend.
- PostgreSQL 15+ is used as the primary database.
- A trained crop disease classification model is available and integrated into the analysis engine.

3. Functional Requirements

3.1 User Management & Authentication

3.1.1 Registration

All users must be able to register on the platform by providing the following information and selecting their role type.

ID	Requirement Description	Priority
FR-UM-01	The system shall allow any user to register by providing full name, email address, phone number, password, and role type (Farmer, Wholesaler, Transporter).	High
FR-UM-02	The system shall validate that email addresses are unique across all registered accounts.	High
FR-UM-03	The system shall encrypt all passwords using bcrypt hashing before storage.	High
FR-UM-04	The system shall send a confirmation notification to the user upon successful registration.	Medium
FR-UM-05	The system shall support registration of all three actor types: FARMER, WHOLESALER, TRANSPORTER, stored as a UserType enum.	High

3.1.2 Authentication

ID	Requirement Description	Priority
FR-UM-06	The system shall authenticate users via email and password, returning a signed JWT token upon success.	High
FR-UM-07	The system shall enforce JWT token expiry and require re-authentication after token expiry.	High
FR-UM-08	The system shall restrict access to all protected API endpoints to authenticated users only.	High
FR-UM-09	The system shall implement Role-Based Access Control (RBAC) so each actor type only accesses permitted features.	High
FR-UM-10	The system shall provide a secure password reset mechanism via email or SMS OTP.	Medium

3.2 Produce Listing & Marketplace

3.2.1 Farmer Listing Management

Farmers must be able to create, update, and manage produce listings that are visible to wholesalers and buyers on the platform marketplace.

ID	Requirement Description	Priority
FR-MK-01	The system shall allow Farmers to create a produce listing specifying crop name, quantity (kg), price per kg, pickup location, and availability status.	High
FR-MK-02	The system shall allow Farmers to update the quantity, price, and status of their existing listings at any time.	High
FR-MK-03	The system shall allow Farmers to delete their own listings.	Medium
FR-MK-04	The system shall automatically associate each listing with the Farmer's user account (user ID).	High
FR-MK-05	The system shall support an updateStatus() method on listings to reflect: Available, Reserved, Sold.	High

3.2.2 Wholesaler Browse & Purchase

ID	Requirement Description	Priority
FR-MK-06	The system shall allow Wholesalers to browse all active produce listings on the marketplace.	High
FR-MK-07	The system shall provide search and filter functionality by crop name, price range, quantity available, and location.	High
FR-MK-08	The system shall allow Wholesalers to express purchase interest on a listing, which notifies the Farmer.	High
FR-MK-09	The system shall allow Wholesalers to request transport for purchased produce, automatically generating a TransportJob linked to the listing.	High
FR-MK-10	The system shall display full Farmer contact information to a Wholesaler only after a confirmed purchase interest has been accepted.	Medium

3.3 Crop Health Analysis

3.3.1 Image Upload & Analysis Request

The crop health analysis module allows Farmers to upload crop images for automated analysis by the C++ OpenCV analysis engine. This is a core differentiating feature of the platform that provides modern AI-powered agronomic guidance.

ID	Requirement Description	Priority
FR-CA-01	The system shall allow authenticated Farmers to upload crop images in JPEG or PNG format via the React Native mobile app.	High
FR-CA-02	The system shall accept image uploads via the POST /api/v1/analyze endpoint using multipart/form-data encoding.	High
FR-CA-03	The C++ API shall validate the JWT authentication token before processing any image analysis request.	High

ID	Requirement Description	Priority
FR-CA-04	The system shall pass the raw binary image data to the ImageProcessor C++ component (OpenCV) for analysis.	High
FR-CA-05	The C++ ImageProcessor shall extract texture features from the submitted crop image using OpenCV algorithms.	High
FR-CA-06	The system shall run a trained prediction model on the extracted texture features to classify crop health status.	High
FR-CA-07	The system shall return a health status result (e.g., Healthy, Diseased, Nutrient Deficiency) along with a confidence score (0.0–1.0).	High
FR-CA-08	The system shall save each analysis result to the CropAnalysis table in PostgreSQL, linked to the requesting Farmer's user ID.	High
FR-CA-09	The React Native frontend shall display the analysis result and relevant agri-recommendations to the Farmer.	High
FR-CA-10	The system shall return the analysis response as a JSON object: { status: 'success', result: 'Healthy', confidence: 0.92, recommendations: [...] }.	High

3.4 Transport Coordination

3.4.1 Transport Job Management

The transport module connects Farmers needing to move produce from farm to market with available Transporters on the platform.

ID	Requirement Description	Priority
FR-TR-01	The system shall allow Farmers or Wholesalers to request a transport job specifying pickup location, destination, cargo description, and desired transport date.	High
FR-TR-02	The system shall associate each TransportJob with a parent Listing (0..1 to many relationship).	High
FR-TR-03	The system shall display all unassigned transport jobs to authenticated Transporters.	High
FR-TR-04	The system shall allow Transporters to accept an available transport job, setting isAssigned = true and linking the Transporter's user ID.	High
FR-TR-05	The system shall execute the matchTransporter() method to notify the job requestor when a Transporter accepts.	High
FR-TR-06	The system shall allow Transporters to update delivery status (e.g., In Transit, Delivered).	Medium
FR-TR-07	The system shall prevent multiple Transporters from accepting the same job (atomic assignment with database lock).	High

3.5 Notification System

ID	Requirement Description	Priority
FR-NT-01	The system shall send in-app push notifications to Farmers when a Wholesaler expresses purchase interest on their listing.	High
FR-NT-02	The system shall notify Farmers and Wholesalers when a Transporter accepts a transport job.	High
FR-NT-03	The system shall notify Farmers when their crop analysis result is ready.	High
FR-NT-04	The system shall support an extensible notification mechanism marked as 'notifies' in the system architecture.	Medium

3.6 Farmer Education Module

ID	Requirement Description	Priority
FR-ED-01	The system shall provide an educational content section accessible to all authenticated Farmers.	Medium
FR-ED-02	The system shall display modern farming techniques including soil preparation, pest control, irrigation, and post-harvest handling.	Medium
FR-ED-03	The system shall link crop analysis results to relevant educational content as recommendations.	Medium
FR-ED-04	The system shall allow administrators to add, update, and remove educational articles.	Low

4.1 Performance Requirements

ID	Requirement Description	Priority
NFR-P-01	The C++ API server (Drogon) shall handle a minimum of 1,000 concurrent API requests per second under normal operating conditions.	High
NFR-P-02	Standard REST API responses (non-image) shall be returned within 200ms under a load of 500 concurrent users.	High
NFR-P-03	The crop image analysis pipeline (upload → OpenCV processing → result) shall complete within 5 seconds for images up to 10MB.	High
NFR-P-04	The mobile application shall load the main marketplace listing page within 3 seconds on a 3G connection.	Medium
NFR-P-05	Database queries for produce listings shall be optimized using indexed columns to return results within 50ms.	Medium

4.2 Security Requirements

ID	Requirement Description	Priority
NFR-S-01	All API communication between the React Native frontend and C++ backend shall use TLS 1.2 or higher (HTTPS only).	High
NFR-S-02	All API endpoints (except /register and /login) shall require a valid JWT bearer token.	High
NFR-S-03	User passwords shall be hashed using bcrypt with a minimum cost factor of 12.	High
NFR-S-04	The system shall implement SQL injection prevention through parameterized queries in all C++ database access layers.	High
NFR-S-05	Uploaded crop images shall be validated for file type and scanned for malicious content before processing.	High
NFR-S-06	The system shall enforce rate limiting of 100 API requests per minute per authenticated user.	Medium
NFR-S-07	User personal data shall be stored encrypted at rest in the PostgreSQL database.	High

4.3 Usability Requirements

ID	Requirement Description	Priority
NFR-U-01	The React Native mobile interface shall support both English and French languages.	High
NFR-U-02	All primary user flows (listing produce, uploading crop image, requesting transport) shall be completable within 3 taps/screens.	Medium
NFR-U-03	The app shall provide clear visual feedback for all asynchronous operations (loading indicators, success/error states).	Medium
NFR-U-04	The mobile UI shall be functional on screen sizes from 4.7 inches upwards and support both portrait and landscape orientations.	Medium

4.4 Reliability and Availability

ID	Requirement Description	Priority
NFR-R-01	The platform shall maintain 99.5% uptime availability (measured monthly).	High
NFR-R-02	The system shall implement database connection pooling to ensure resilience under peak load.	High
NFR-R-03	The system shall implement graceful error handling for all API endpoints, returning structured JSON error responses.	High
NFR-R-04	Daily automated database backups shall be maintained with a minimum retention period of 30 days.	Medium

4.5 Maintainability & Scalability

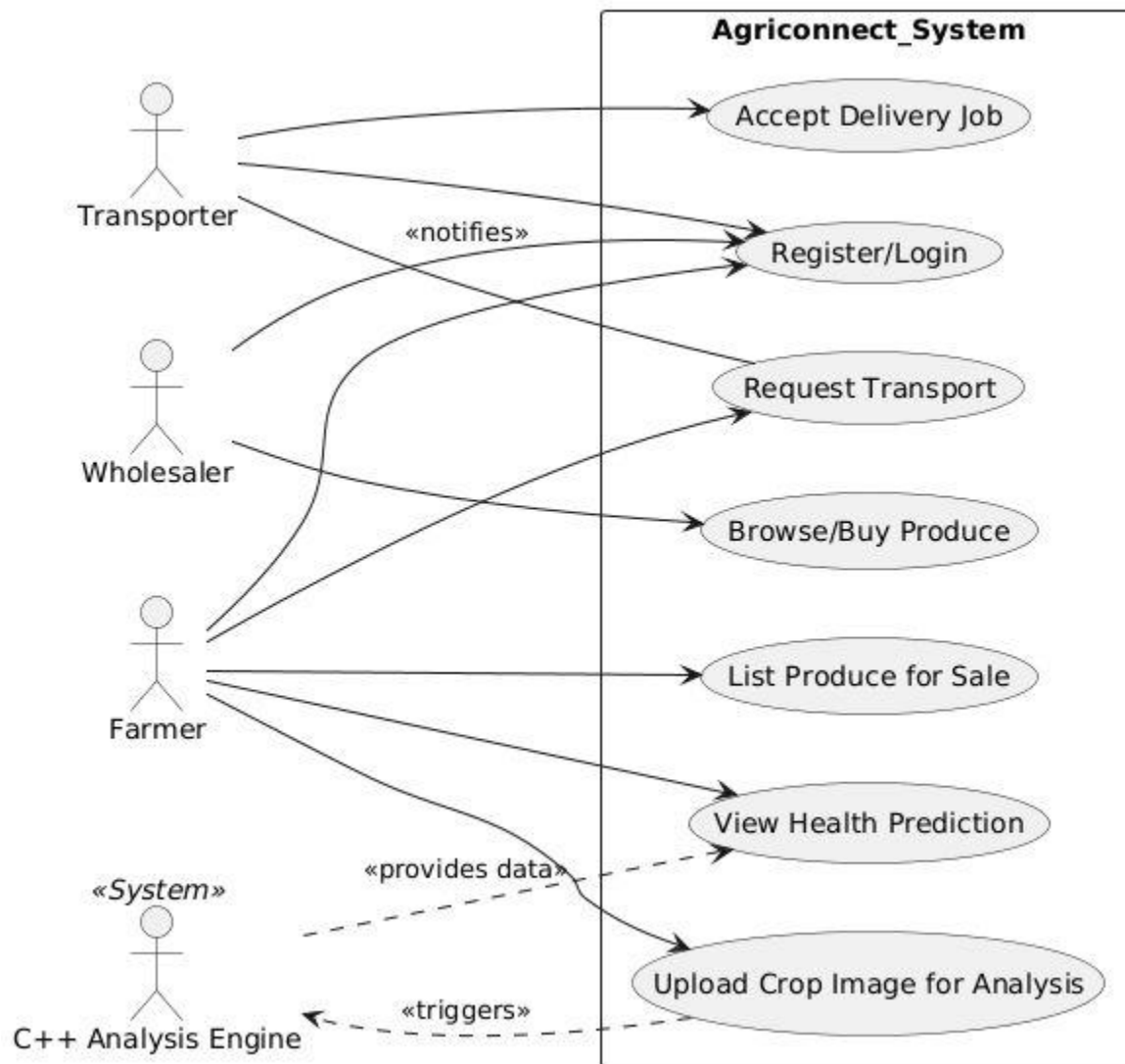
ID	Requirement Description	Priority
NFR-M-01	The C++ backend shall follow a layered architecture: Controller → Service → Repository → Database.	High
NFR-M-02	The system architecture shall support horizontal scaling of the API server behind a load balancer.	Medium
NFR-M-03	All API endpoints shall be versioned (e.g., /api/v1/) to support backward compatibility during updates.	High
NFR-M-04	The image analysis engine shall be modular and replaceable, decoupled from the API controller layer.	Medium

5. UML Diagrams

5.1 Use Case Diagram

The following Use Case Diagram illustrates the interactions between all system actors (Farmer, Wholesaler, Transporter, C++ Analysis Engine) and the primary system use cases within the Agriconnect system boundary.

The diagram is illustrated below:



Use Case	Actor(s)	Description
Register / Login	Farmer, Wholesaler, Transporter	All user types register an account and log in to access the platform. The system sends a notification upon successful authentication.
List Produce for Sale	Farmer	Farmer creates a produce listing with crop name, quantity, price, and pickup location for visibility on the marketplace.
Browse / Buy Produce	Wholesaler	Wholesaler views available listings, applies filters, and expresses purchase interest on selected produce.
Request Transport	Farmer, Wholesaler	A transport job is created specifying pickup location and destination to move produce from farm to market.
Accept Delivery Job	Transporter	Transporter views unassigned transport jobs and accepts one, marking it as assigned and triggering a notification.
Upload Crop Image for Analysis	Farmer	Farmer uploads a crop image which triggers the C++ Analysis Engine via the API. This is a system-actor interaction.
View Health Prediction	Farmer	Farmer receives and views the health status result (Healthy / Diseased) and confidence score provided by the Analysis Engine.

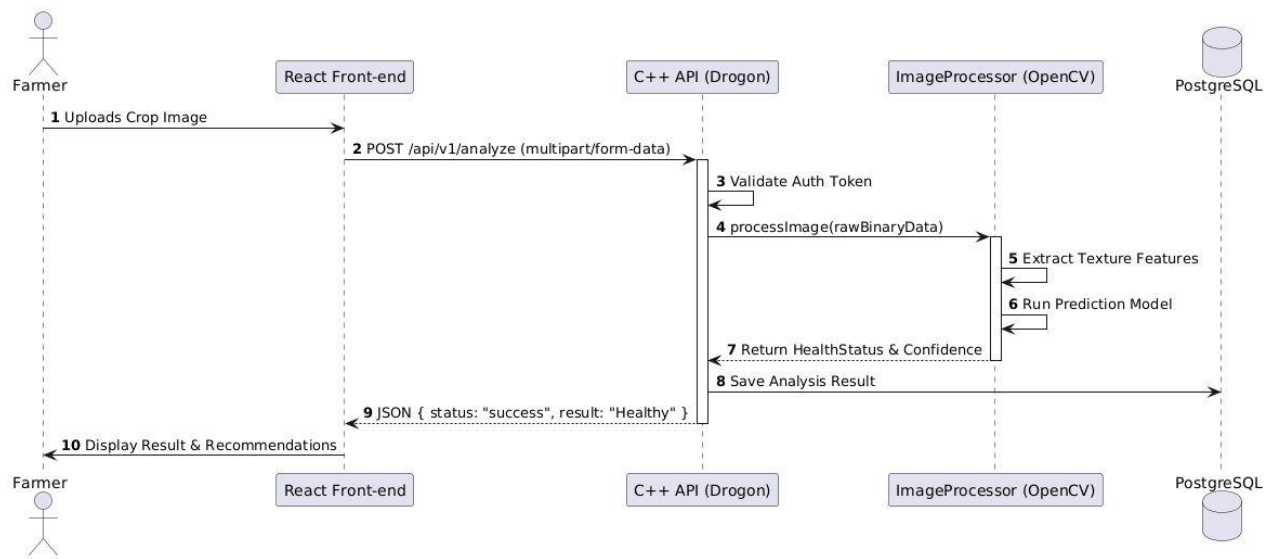
Key UML Relationships in the Use Case Diagram:

- <<notifies>> — The Register/Login use case triggers a system notification to the relevant actor upon completion.
- <<provides data>> — The C++ Analysis Engine (System actor) provides processed health prediction data back to the View Health Prediction use case.
- <<triggers>> — The Upload Crop Image use case triggers execution of the C++ Analysis Engine.

5.2 Sequence Diagram — Crop Analysis Flow

The sequence diagram below documents the complete end-to-end message flow for the Crop Health Analysis use case, from the Farmer uploading an image through to displaying the result on the React Native frontend.

The diagram is illustrated below:

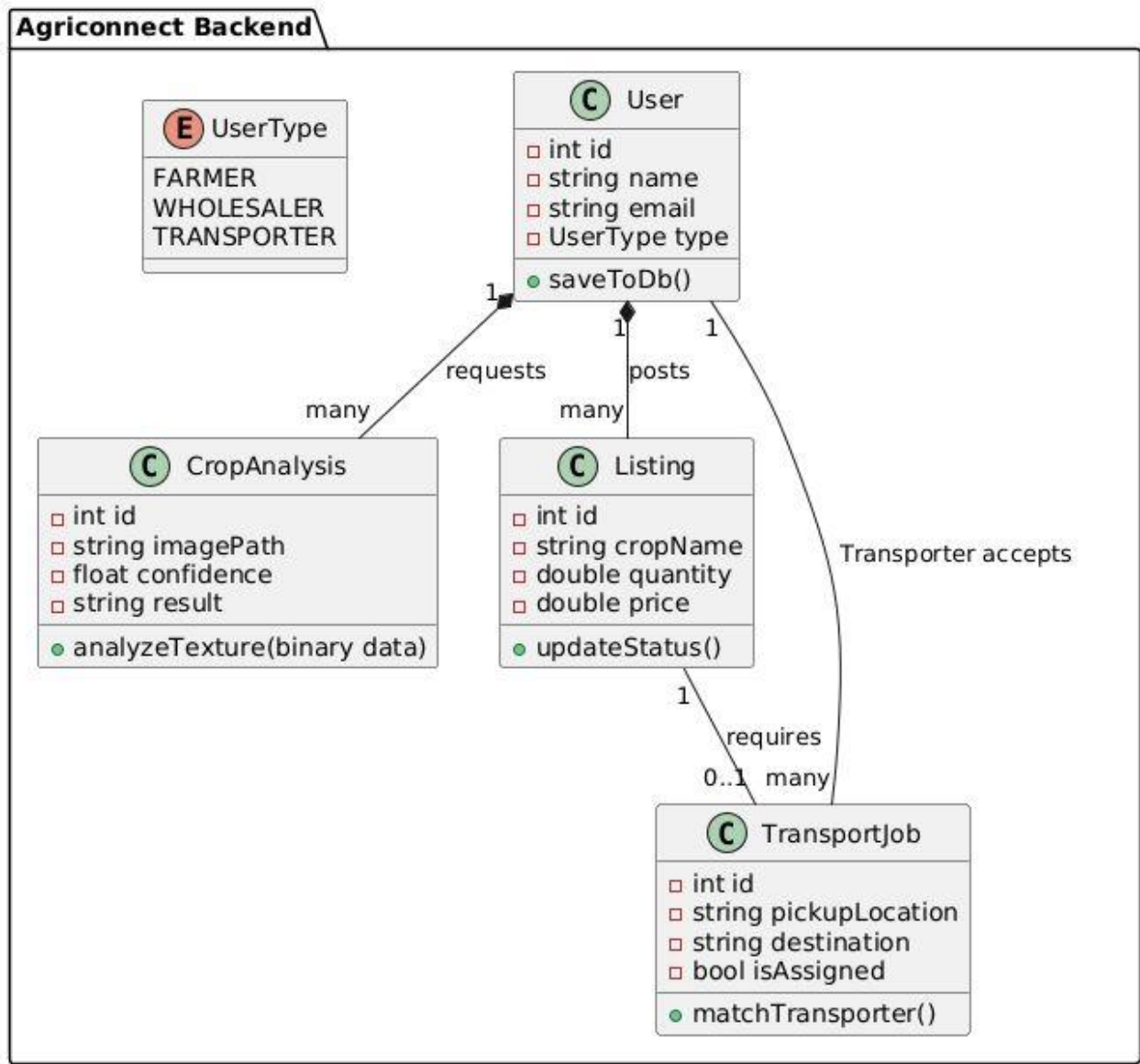


Step	From	To	Message / Action
1	Farmer	React Native Frontend	Uploads crop image via mobile UI
2	React Native Frontend	C++ API (Drogon)	POST /api/v1/analyze (multipart/form-data)
3	C++ API (Drogon)	C++ API (Drogon)	Validate Auth Token (JWT verification, self-loop)
4	C++ API (Drogon)	ImageProcessor (OpenCV)	processImage(rawBinaryData)
5	ImageProcessor (OpenCV)	ImageProcessor (OpenCV)	Extract Texture Features (self-loop)
6	ImageProcessor (OpenCV)	ImageProcessor (OpenCV)	Run Prediction Model (self-loop)
7	ImageProcessor (OpenCV)	C++ API (Drogon)	Return HealthStatus & Confidence (e.g., 'Healthy', 0.92)
8	C++ API (Drogon)	PostgreSQL	Save Analysis Result (INSERT CropAnalysis record)
9	C++ API (Drogon)	React Native Frontend	JSON { status: 'success', result: 'Healthy' }
10	React Native Frontend	Farmer	Display Result & Recommendations

5.3 Class Diagram — Agriconnect Backend

The class diagram defines the object-oriented model of the C++ backend, showing all primary domain classes, their attributes, methods, and inter-class relationships.

The diagram is illustrated below:



5.3.1 Enum: UserType

Enum	Values
UserType	FARMER WHOLESALER TRANSPORTER

5.3.2 Class Definitions

Class	Attributes	Methods	Relationships
User	int id string name string email UserType type	saveToDb()	1 User posts many Listings 1 User requests many CropAnalyses 1 User (Transporter) accepts many TransportJobs
Listing	int id string cropName double quantity double price	updateStatus()	Many Listings belong to 1 User (Farmer) 1 Listing requires 0..1 TransportJob
CropAnalysis	int id string imagePath float confidence string result	analyzeTexture(binary data)	Many CropAnalyses belong to 1 User (Farmer)
TransportJob	int id string pickupLocation string destination bool isAssigned	matchTransporter()	Many TransportJobs linked to 1 Listing Many TransportJobs accepted by 1 User (Transporter)

5.3.3 Class Relationships Summary

Relationship	Multiplicity	Description
User → Listing	1 to many	A Farmer (User) can post multiple produce listings
User → CropAnalysis	1 to many	A Farmer can request multiple crop analyses
User (Transporter) → TransportJob	1 to many	A Transporter can accept multiple transport jobs
Listing → TransportJob	1 to 0..many	A listing may require one or more transport jobs

6. System Architecture

6.1 Architectural Overview

Agriconnect follows a three-tier client-server architecture consisting of a React Native mobile presentation layer, a C++ Drogon-based application layer, and a PostgreSQL data layer. The image analysis subsystem is integrated as a native C++ module within the application layer.

Layer	Technology	Responsibility
Presentation Layer	React Native (iOS & Android)	User interface, state management, API calls, image capture, result display
Application Layer	C++ with Drogon Framework	REST API routing, business logic, JWT authentication, request handling
Image Analysis Module	C++ with OpenCV 4.8+	Texture feature extraction, ML model inference, health classification
Data Layer	PostgreSQL 15+	Persistent storage for users, listings, crop analyses, transport jobs
Communication Protocol	HTTPS / REST / JSON	All client-server communication over TLS-secured HTTP/1.1 or HTTP/2

6.2 Backend Architecture (C++ Drogon)

The C++ backend follows a layered MVC-inspired architecture:

- **Controller Layer** Drogon HTTP controllers that route incoming requests to service methods. Handles request validation and response serialization.
- **Service Layer** Business logic layer. Orchestrates calls between controllers, the ImageProcessor, and the repository layer.
- **Repository Layer** C++ data access objects (DAOs) that construct and execute parameterized SQL queries against PostgreSQL.
- **ImageProcessor Module** Standalone C++ class using OpenCV. Accepts raw binary image data, performs texture analysis, and executes the trained classification model.

6.3 Frontend Architecture (React Native)

- **Navigation** — React Navigation with stack and tab navigator for role-based screen routing.
- **State Management** — React Context API or Redux Toolkit for global state (auth token, user profile, listings).
- **API Layer** — Axios or Fetch API for REST communication with the Drogon backend.
- **Image Handling** React Native Camera / Image Picker for crop image capture and upload.
- **Offline Cache** AsyncStorage for local caching of recent listings and analysis results.

6.4 API Endpoint Summary

Method	Endpoint	Auth	Actor	Description
POST	/api/v1/register	None	All	Register a new user account
POST	/api/v1/login	None	All	Authenticate and receive JWT token
GET	/api/v1/listings	JWT	All	Get all active produce listings
POST	/api/v1/listings	JWT	Farmer	Create a new produce listing
PUT	/api/v1/listings/:id	JWT	Farmer	Update an existing listing
DELETE	/api/v1/listings/:id	JWT	Farmer	Delete a listing
POST	/api/v1/analyze	JWT	Farmer	Upload crop image for health analysis
GET	/api/v1/analyses	JWT	Farmer	Get Farmer's past crop analyses
GET	/api/v1/transport/jobs	JWT	Transporter	Get all unassigned transport jobs
POST	/api/v1/transport/jobs	JWT	Farmer / Wholesaler	Create a new transport job
PUT	/api/v1/transport/jobs/:id/accept	JWT	Transporter	Accept a transport job
GET	/api/v1/education	JWT	Farmer	Get educational farming content

7. Data Requirements

7.1 Database Schema Overview

7.1.1 users Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment user identifier
name	VARCHAR(255)	NOT NULL	Full name of the user
email	VARCHAR(255)	NOT NULL, UNIQUE	Email address (used for login)
password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed password
type	user_type_enum	NOT NULL	FARMER WHOLESALER TRANSPORTER
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

7.1.2 listings Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment listing identifier
user_id	INTEGER	FK → users.id NOT NULL	The Farmer who created the listing
crop_name	VARCHAR(255)	NOT NULL	Name of the crop (e.g., Maize, Cassava)
quantity	DOUBLE PRECISION	NOT NULL	Available quantity in kilograms
price	DOUBLE PRECISION	NOT NULL	Price per kilogram
pickup_location	VARCHAR(255)	NOT NULL	Farm location for pickup
status	listing_status_enum	DEFAULT 'Available'	Available Reserved Sold
created_at	TIMESTAMP	DEFAULT NOW()	Listing creation timestamp

7.1.3 crop_analyses Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment analysis identifier
user_id	INTEGER	FK → users.id NOT NULL	The Farmer who submitted the image
image_path	VARCHAR(500)	NOT NULL	Server-side path or object store URL for the image
confidence	FLOAT	NOT NULL	Model confidence score (0.0 to 1.0)

result	VARCHAR(100)	NOT NULL	Classification result (e.g., Healthy, Diseased)
analyzed_at	TIMESTAMP	DEFAULT NOW()	Timestamp of analysis completion

7.1.4 transport_jobs Table

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Auto-increment transport job identifier
listing_id	INTEGER	FK → listings.id, NULLABLE	Associated produce listing (optional)
pickup_location	VARCHAR(255)	NOT NULL	Pickup address/location
destination	VARCHAR(255)	NOT NULL	Delivery destination
is_assigned	BOOLEAN	DEFAULT FALSE	Whether a Transporter has accepted this job
transporter_id	INTEGER	FK → users.id, NULLABLE	Assigned Transporter (set on acceptance)
created_at	TIMESTAMP	DEFAULT NOW()	Job creation timestamp

8. External Interface Requirements

8.1 User Interfaces

- The React Native mobile application shall serve as the primary user interface for all actors.
- The app shall support both Android (API level 26+) and iOS (14+) platforms.
- All screens shall follow Material Design 3 principles adapted for agricultural users.
- The dashboard home screen shall adapt its visible modules based on the authenticated user's role type.

8.2 Hardware Interfaces

- Camera — The app shall use the device camera to capture crop images for health analysis (Android and iOS camera API via React Native Camera or Expo Camera).
- Storage — The app shall use device local storage for offline caching of listings and results.
- GPS/Location — The app may optionally use device GPS to auto-fill pickup location fields for Farmers creating listings.

8.3 Software Interfaces

Interface	Technology	Purpose
Mobile-to-API	HTTPS REST JSON over Axios	All React Native communication with C++ Drogon backend
API-to-Database	libpqxx (C++ PostgreSQL client)	C++ backend database queries and transactions
API-to-ImageProcessor	C++ function call (in-process)	Drogon controller invokes OpenCV ImageProcessor class
Push Notifications	Firebase Cloud Messaging (FCM)	Cross-platform push notifications to mobile devices
File Storage	Local filesystem or S3-compatible store	Persistent storage for uploaded crop images

9. Testing & Quality Assurance Requirements

9.1 Unit Testing

- All C++ backend service and repository classes shall have unit tests with minimum 80% code coverage.
- The CropAnalysis::analyzeTexture() method shall be tested with known input images and expected output classifications.
- React Native components shall be unit tested using Jest and React Native Testing Library.

9.2 Integration Testing

- The full image analysis pipeline (upload → OpenCV → prediction → database save → JSON response) shall be integration tested end-to-end.
- All API endpoints shall be tested using automated REST API testing (Postman / Newman or custom C++ test harness).
- Database transaction integrity shall be tested for concurrent transport job acceptance scenarios.

9.3 User Acceptance Testing

- A representative group of Farmers, Wholesalers, and Transporters shall participate in UAT in a staging environment.
- UAT shall validate all high-priority functional requirements defined in Section 3.
- Performance benchmarks in NFR Section 4.1 shall be validated using load testing tools (e.g., Apache JMeter).

10. Constraints, Compliance & Regulatory Requirements

10.1 Technical Constraints

- Backend language: C++ (mandatory — Drogon framework)
- Frontend: React Native (cross-platform iOS + Android)
- Image processing: OpenCV (no cloud-based ML services for core analysis)
- Database: PostgreSQL only

10.2 Regulatory Compliance

- The system shall comply with applicable data protection legislation in target operating countries (e.g., Cameroon's Law No. 2010/012 on cybersecurity and cybercrime).
- User personal data (name, email, phone) shall not be shared with third parties without explicit user consent.
- All data transmissions shall use TLS encryption to protect personal information in transit.

11. Appendices

Appendix A — Use Case Reference Summary

UC ID	Use Case Name	Primary Actor	Precondition	Postcondition
UC-01	Register / Login	All Actors	None	User authenticated with JWT token
UC-02	List Produce for Sale	Farmer	Authenticated as Farmer	New listing active on marketplace
UC-03	Browse / Buy Produce	Wholesaler	Authenticated as Wholesaler	Purchase interest sent to Farmer
UC-04	Request Transport	Farmer / Wholesaler	Authenticated user	New TransportJob created
UC-05	Accept Delivery Job	Transporter	Authenticated as Transporter	Job isAssigned = true; parties notified
UC-06	Upload Crop Image	Farmer	Authenticated as Farmer	CropAnalysis record created; analysis queued
UC-07	View Health Prediction	Farmer	UC-06 completed	Farmer views result and recommendations

Appendix B — Revision History

Version	Date	Author	Description
1.0	April 2026	Agriconnect Dev Team	Initial SRS document — full requirements specification based on approved UML diagrams and system design.

Appendix C — References

1. Drogon C++ Web Framework Documentation — <https://drogon.org>
2. OpenCV 4.8 Documentation — <https://docs.opencv.org>
3. React Native Documentation — <https://reactnative.dev>
4. PostgreSQL 15 Documentation — <https://www.postgresql.org/docs/>
5. Agriconnect Use Case Diagram (Image 1 — provided)
6. Agriconnect Sequence Diagram: Crop Analysis Flow (Image 2 — provided)
7. Agriconnect Backend Class Diagram (Image 3 — provided)

— End of Agriconnect Software Requirements Specification v1.0 —